Rubrica: Systems Design

Learning a black box, one proven rule at a time

Reto 2 · La caja negra · rubrica.dev · Kevin Vicent · v11

The black box is an unknown decision procedure. We find its rules with experiments. We prove each rule against the real service. Then we compile the rules into a deterministic replica. Most agent systems need an LLM judge. Our system has an exact judge: the original service.

1. Problem

A legacy **ledger service** receives text messages on `POST /msg`. The body is `{"m": "CMD;arg…;control"}`. The commands are `PING`, `ALTA`, `CONSULTA`, `DEPOSITO`, `TRANSFER` and `ANULA`. The service also has `/__reset` and `/health`.

We must build a replacement. After `/__reset`, each response must have **the same status and body** as the original. The tests do not compare headers or JSON key order. There are **100 hidden tests**: 40 single requests, 45 sequences of 2–5 steps, 15 sequences of 6–12 steps. Every quirk is part of the contract (Hyrum's Law): `"recieved"`, `"comando desconocido "`, `cuentaId` vs. `cuenta_id`. The replica must not call the original. **The service changes during the day. The tests use the 20:00 version.**

ID	Assumption	Status	How we test it
A1	The output depends only on state and request	open	Replay one sequence twice after reset
A2	Each token has its own state; `/__reset` restores `AC-0001`, `OP-0001`	README	Reset, then repeat a sequence
A3	The control is an integer checksum on the arguments only (15 values seen, range 3–98)	hypothesis	Fit offline, then targeted probes
A3b	The control is checked before the account lookup	**proven**	Bad control + missing account → E202
A4	Tests are sequential	README	—

Hard cases: the control formula · auth (team token, other token, no token) · **error precedence** (E2xx integrity, E301/E302 syntax, E401 lookup, E506/E507 business) · ID sequences · `ANULA` twice · limits · unknown routes · **drift**.

Out of scope: headers (the tests do not compare them) · latency under the 5 s step limit.

3. How we measure: evals first

We optimize against evals and protect them from Goodhart's Law: **corpus** = train · **held-out fuzz** = test · **canary** = "is the eval still true?" · **hidden set** = final test.

- **M1 Exact match** (official comparator): 100% on the corpus (versioned per service version) and the 20 `ejemplos`; **≥ 95% on held-out fuzz**. We never fix code against held-out fuzz.
- **M2 Coverage:** command × precondition × error matrix. Low-confidence rules and empty cells first. **Budget: 8,000 requests/day, 10/s per token.** Plan (566 already spent): 1,435 discovery · 2,500 fuzz · 480 canary · 3,019 reserve.
- **M3 Drift:** a 10-request canary every 10 min (480 requests, 12:00–20:00). Full replay only if it fails. Detect in ≤ 10 min, fix in ≤ 45 min.
- **Checksum eval:** predict all logged controls (train) and new strings with digits, lowercase and accents (test).
- **Seeded tests:** each error alone and **in pairs**, control ±1, case variants, arity ±1, `ANULA` twice, transfers near the limit.
- **3 validations:** only if M1 did not decrease (~13:30, ~17:30, ~19:30).

2. Architecture: dual-process, one ontology

1. **Agent harness (System 2, offline).** A person and a reasoning model use only recorded tools: `probe`, `sequence`, `fuzz`, `shrink`, `diff`. Each request becomes one line of `experimentos.jsonl`, with its hypothesis and `X-Req`. **Rule: no request without a prediction.** A code gate (a planned `beforeToolCall` hook) blocks a request if it has no prediction, if the log already predicts it, or if it exceeds the budget.
2. **Hypothesis search, not answer search.** Our first sweep tried controls 0–99 and used 543 requests. Now the harness fits candidate rules offline, at zero cost. Then it sends the request that eliminates the most candidates. **Target: one formula in fewer than 50 probes.**
3. **Service ontology (memory).** Each rule is typed: `Rule{command, when, then, effect, evidence, confianza}`. The checksum is a typed slot: `Checksum{scope, mapping, base, modulus, transform}`. One table does three jobs: the replica **executes** it, `SPEC.md` is **generated** from it (7 sections, `confirmado / probable / supuesto`), and the tests come from its evidence.
4. **Replica (System 1, runtime).** HTTP adapter → serial queue → decision core `handle(state[token], msg) → (state, resp)`. Order (we still test this precedence): route → auth → parse → control → command → arity → lookup → business rule → effect. State is in memory.

Decision	Chosen	Rejected, and why
Runtime	**Compiled rule table:** System 2 knowledge made automatic; no probabilities at runtime	*A probabilistic layer (LLM or classifier) feeding the rules, as natural-language systems need:* the input is a formal protocol, so there is nothing to perceive; not exact, not repeatable. *Record and replay:* fails on new inputs.
Discovery	**A person and a model in a harness**, exact oracle	*Free LLM chat:* no proof, no log. *Autonomous agent:* false spec claims lose points. *Manual curl:* too slow.
Knowledge	**Typed ontology**, one source for code, spec and tests	*Separate spec, code and tests:* they diverge after the change. *RAG over logs:* approximate recall.
Checks	**Code for all exact checks, cheapest first:** cache → recorded corpus → live original (budget gate, checksum, rule replay). Drift triage in code; a **System One classifier** only if code cannot separate drift from noise. Fails closed.	*LLM judge as verifier:* false passes. The original is a free, exact judge.
Server and hosting	**Minimal server we control** (exact bodies, also for unknown routes); **one always-on instance**	*Framework defaults:* different 404/405 bodies. *Autoscaling:* each instance has its own state.

Risks: formula outside our family (add candidates, probe again) · budget runs out (cache, never repeat) · late drift (canary runs until 20:00).